

Projektarbeit m294

Projekttitle: Sport-Challenge Tracker Frontend

Datum: 2026

Name: Bürgin Joris

Klasse: 24-G

Ausgangslage

Die Projektarbeit ist ein zentraler Bestandteil dieses Moduls. Ziel ist es, ein Angular Frontend für die im Modul 295 entwickelte Backend-Applikation zu erstellen. Die eingesetzten Technologien orientieren sich am Demo-Projekt und können durch weitere Open-Source-Pakete ergänzt werden.

Projektbeschreibung: Sport-Challenge Tracker Frontend

Die Applikation ermöglicht es Benutzern, persönliche Sport-Challenges zu verwalten und ihren Fortschritt zu verfolgen. Das Frontend verbindet sich mit der bestehenden Spring Boot REST API aus Modul 295.

Ein Benutzer kann beispielsweise folgende Challenges erstellen:

- 100 km Mountainbike fahren
- 10 Gym Workouts absolvieren
- 5 Skitage erreichen

Zu jeder Challenge können Progress Entries erfasst werden, welche den Fortschritt repräsentieren. Die Applikation zeigt den aktuellen Fortschritt an und markiert eine Challenge automatisch als COMPLETED, sobald der Zielwert erreicht ist.

Zusätzlich können Challenges Kategorien zugewiesen werden, zum Beispiel:

- MountainBike
- Gym
- Ski

Funktionale Anforderungen

- Benutzer können sich über Keycloak einloggen und ausloggen
- Challenges anzeigen, erstellen, bearbeiten und löschen
- Fortschritte (Progress Entries) zu einer Challenge erfassen
- Automatische Anzeige des Fortschritts und Status (OPEN / COMPLETED)
- Kategorien anzeigen (alle Benutzer) und verwalten (nur Admin)
- Rollenbasierter Zugriff: USER und ADMIN

Technische Umsetzung

Die Anwendung wird mit Angular als Single Page Application (SPA) umgesetzt und kommuniziert über HTTP mit der REST API des Backends.

Verwendete Technologien:

- Angular (CLI)
- Angular Material

- Angular Router (Lazy Loading)
- Reactive Forms
- HTTP Client
- angular-oauth2-oidc (Keycloak / OAuth2)
- Vitest (Unit Tests)
- Git

Komponenten

Die Applikation besteht aus Pages (vollständige Ansichten) und Einzelkomponenten (wiederverwendbare UI-Teile).

Pages

Komponente	Route	Beschreibung
LoginPage	/login	Keycloak Login, Weiterleitung nach erfolgreichem Login
NoAccessPage	/no-access	Wird angezeigt wenn eine Route ohne Berechtigung aufgerufen wird
ChallengeListPage	/challenges	Übersicht aller Challenges mit Filterfunktion
ChallengeDetailPage	/challenges/:id	Detailansicht einer Challenge inkl. Progress Entries (Lazy Loading)
ChallengeFormPage	/challenges/new, /challenges/:id/edit	Formular zum Erstellen und Bearbeiten einer Challenge (kombiniert)
CategoryListPage	/categories	Übersicht und Verwaltung aller Kategorien (nur ADMIN)
CategoryFormPage	/categories/new, /categories/:id/edit	Formular zum Erstellen und Bearbeiten einer Kategorie (nur ADMIN)

Einzelkomponenten

Komponente	Beschreibung
NavBarComponent	Navigationsleiste mit Links, Login/Logout-Button und Rollenanzeige
ChallengeCardComponent	Challenge-Karte mit Titel, Status, Fortschrittsbalken und Kategorie
ProgressEntryListComponent	Liste aller Progress Entries einer Challenge
ProgressEntryFormComponent	Formular zum Erfassen eines neuen Progress Entry

Total: 7 Pages + 4 Einzelkomponenten = 11 Komponenten (Mindestanforderung: 8)

Models

Die folgenden TypeScript Models spiegeln die Entitäten des Backends wider:

Model	Felder
challenge.ts	id, title, description, targetValue, currentValue, status, startDate, endDate, categoryId
progress-entry.ts	id, value, date, note, challengeId
category.ts	id, name

Services

Jeder Service ist fuer eine Ressource der REST API zustaeendig und kapselt alle HTTP-Aufrufe.

Service	Verwendete Endpoints
ChallengeService	GET/POST /api/challenges, GET/PUT/DELETE /api/challenges/:id
ProgressService	GET /api/progress/by-challenge/:id, POST /api/progress
CategoryService	GET/POST /api/categories, GET/PUT/DELETE /api/categories/:id
AuthService	Token-Verwaltung, Login/Logout, Rollenabfrage ueber Keycloak

Routing

Das Routing wird mit dem Angular Router umgesetzt. Die ChallengeDetailPage verwendet Lazy Loading. Alle gesicherten Routen sind durch Guards geschützt.

Route	Komponente	Guard / Bemerkung
/login	LoginPage	Öffentlich zugänglich
/no-access	NoAccessPage	Öffentlich zugänglich
/challenges	ChallengeListPage	AuthGuard (eingeloggt)
/challenges/new	ChallengeFormPage	AuthGuard (UPDATE-Rolle)
/challenges/:id	ChallengeDetailPage	AuthGuard, Lazy Loading
/challenges/:id/edit	ChallengeFormPage	AuthGuard (UPDATE-Rolle)
/categories	CategoryListPage	AuthGuard (ADMIN-Rolle)
/categories/new	CategoryFormPage	AuthGuard (ADMIN-Rolle)
/categories/:id/edit	CategoryFormPage	AuthGuard (ADMIN-Rolle)

Security

Die Applikation ist über OAuth2 mit Keycloak abgesichert, identisch zur Konfiguration des Backends aus Modul 295.

Keycloak Konfiguration

- Realm: challengetracker
- Client: challengetracker
- Port: 8080

Benutzer

Admin Benutzer

- Username: Joris
- Passwort: admin123
- Rollen: ADMIN, READ, UPDATE

Normaler Benutzer

- Username: user
- Passwort: admin123
- Rollen: READ, UPDATE

Rollen

USER (READ + UPDATE)

- Challenges anzeigen, erstellen, bearbeiten und löschen
- Progress Entries erfassen
- Kategorien anzeigen

ADMIN

- Alle Rechte des USER
- Kategorien erstellen, bearbeiten und löschen
- Zugriff auf den Admin-Bereich

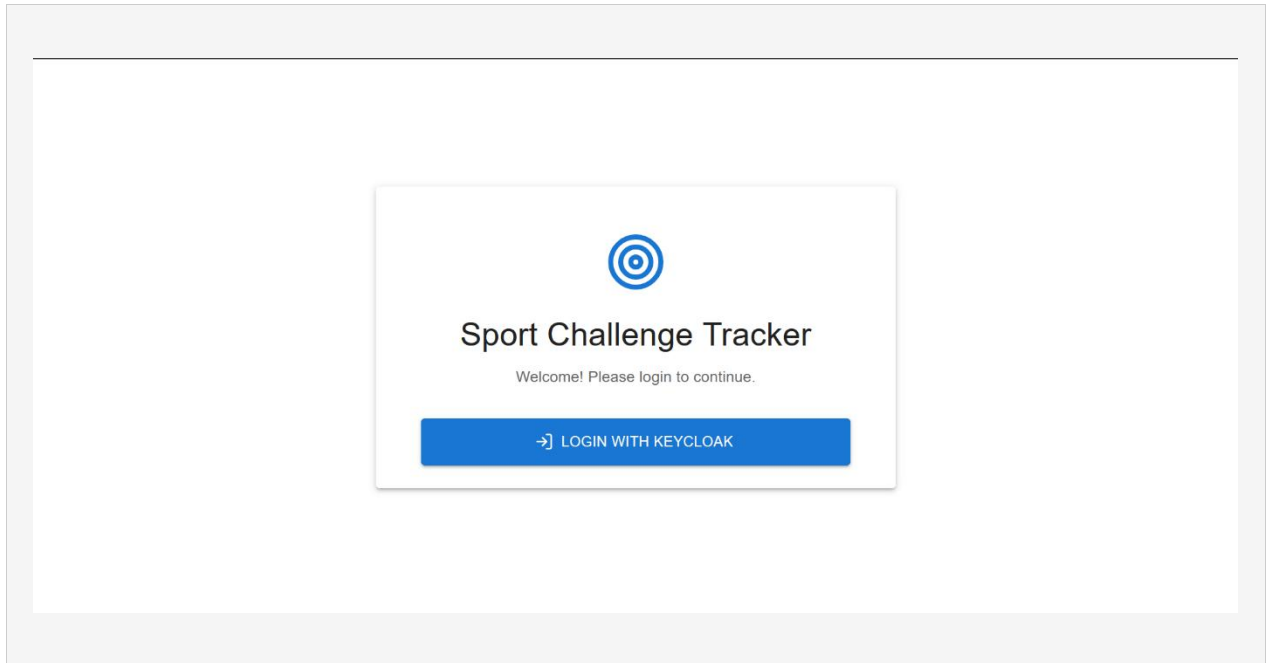
Umsetzung im Frontend

- AuthGuard: Schützt Routen vor nicht eingeloggten Benutzern
- RoleGuard: Prüft ob der Benutzer die benötigte Rolle besitzt
- IsInRoleDirective: Zeigt oder versteckt UI-Elemente basierend auf der Rolle
- AppAuthService: Verwaltet Tokens, Login/Logout und Rollenabfrage

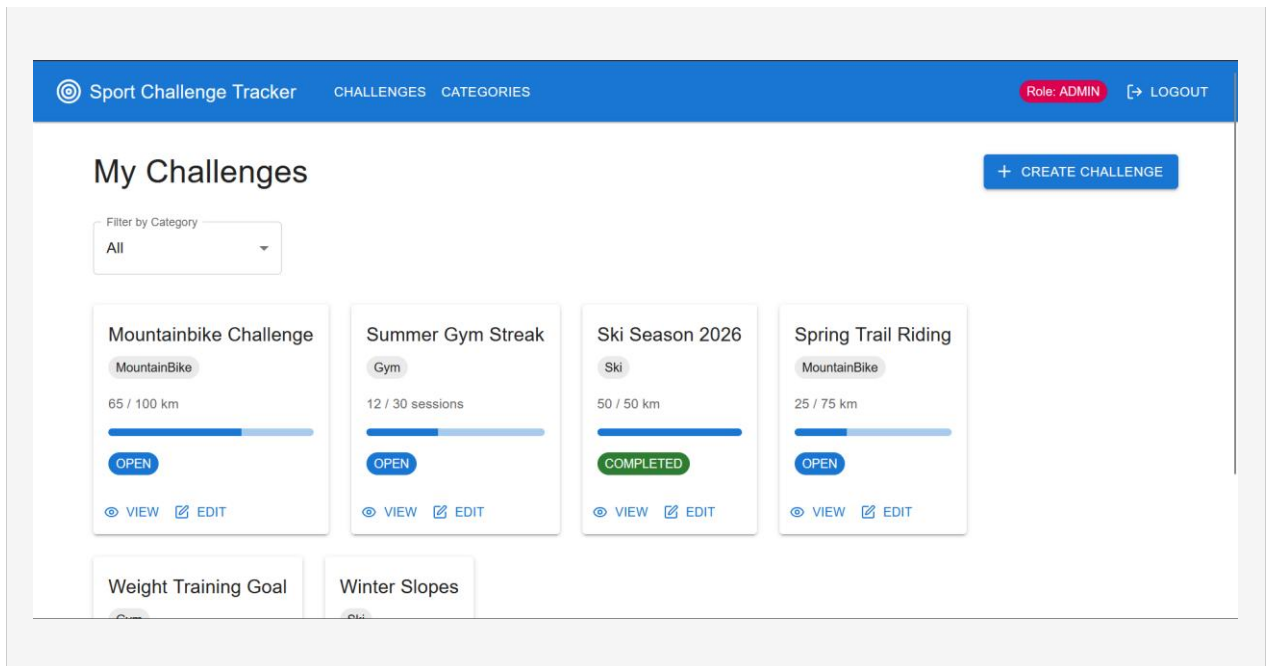
Wireframes

Die folgenden Wireframes zeigen die wichtigsten Ansichten der Applikation.

Login-Seite



Challenge-Liste



Challenge-Detail

Sport Challenge TrackerCHALLENGESCATEGORIESRole: ADMINLogout

[← BACK TO CHALLENGES](#)

Mountainbike Challenge

Complete 100km of mountain biking trails during the summer season. Track your progress and achieve your stress goal!

MountainbikeOPEN

Progress65 / 100 km65.0% complete

Progress Entries

Date	Value	Note
2026-06-01	25	Morning ride at Forest Trail
2026-05-28	15	Evening ride
2026-05-25	25	Weekend mountain trail

Add Progress Entry

Value *

Note

+ ADD PROGRESS

Challenge-Formular

Sport Challenge TrackerCHALLENGESCATEGORIESRole: ADMINLogout

[← BACK TO CHALLENGES](#)

Create New Challenge

Title *

Description *

Target Value *

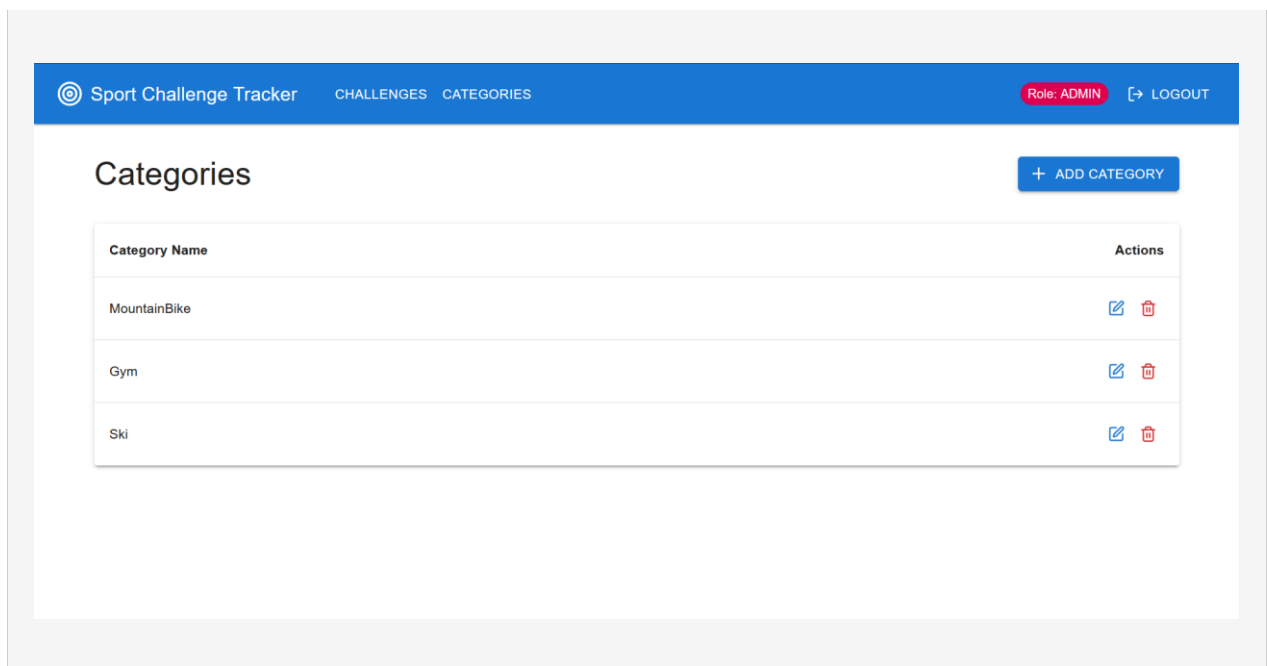
Start Date *TT . MM . JJJJ

End Date *TT . MM . JJJJ

Category *

SAVEX CANCEL

Kategorie-Verwaltung (Admin)



Testing

Die Anwendung wurde mit Vitest getestet. Implementiert wurden Komponenten-Tests und Service-Tests.

Test-Datei	Typ	Was wird getestet
challenge.service.spec.ts	Service-Test	HTTP-Calls, CRUD-Operationen, Fehlerbehandlung
category.service.spec.ts	Service-Test	GET/POST/PUT/DELETE Kategorien
challenge-list.spec.ts	Komponenten-Test	Rendering der Challenge-Liste, Filterlogik
challenge-detail.spec.ts	Komponenten-Test	Rendering der Details und Progress Entries

Tests können mit folgendem Befehl ausgeführt werden:

- `npx vitest run`

Anwendung starten

1. PostgreSQL starten

Datenbank `challengetrackerdb` muss laufen.

2. Keycloak starten

Keycloak auf Port 8080 starten, Realm challengetracker importieren.

3. Backend starten

Spring Boot Backend auf Port 9090 starten:

- `mvn spring-boot:run`

4. Frontend starten

- `npm install`
- `ng serve`

Die Applikation ist anschliessend erreichbar unter:

- `http://localhost:4200`

Bewertungskriterien Umsetzung

Die Anforderungen werden wie folgt erfüllt:

- Angular Frontend: Umsetzung als Single Page Application
- Mindestens 8 Komponenten: 7 Pages + 4 Einzelkomponenten = 11 total
- REST API Anbindung: 4 Services (Challenge, Progress, Category, Auth)
- CRUD: Vollständig implementiert für Challenges und Kategorien (Admin)
- OAuth2 + Keycloak: Absicherung über angular-oauth2-oidc
- Rollen: USER (READ + UPDATE) und ADMIN
- Routing: Angular Router mit Lazy Loading und Guards
- Unit Tests: Vitest, Service- und Komponenten-Tests
- Git: Versionsverwaltung auf GitHub